OmniAgentAI: A Neuro-Symbolic Multi-Agent Agentic RAG Architecture with Tree-of-Thought Planning, ReAct Tool Use, Crew Evaluation, and Self-Verification

Draft manuscript prepared for ResearchGate-style sharing

Abstract

OmniAgentAI is a proposed multi-agent agentic artificial intelligence platform designed to solve heterogeneous user tasks through a combination of language-model reasoning, routing, retrieval, tool execution, multi-agent workflows, and verification. The system begins with a direct language-model capability check: if the query can be answered safely and verifiably by the model, the system may respond directly; otherwise, it escalates the query to a router organized as the root of an agent tree. Leaf and parent agents specialize in domains such as coding, calculator tasks, weather, banking, travel planning, healthcare, research, and general knowledge. Depending on task type, agents use Chain-of-Thought style staged reasoning, Tree-of-Thought search over candidate strategies, ReAct loops for tool use, Retrieval-Augmented Generation, Model Context Protocol integrations, local databases, and CrewAI-style multi-role evaluation. The architecture adds verification, self-correction, memory, and future extensions including multimodal reasoning, theorem proving with Lean, algorithm discovery agents, AI scientist workflows, multi-LLM collaboration, distributed agent ecosystems, and autonomous research systems. This paper presents the conceptual architecture, workflow, component design, safety strategy, evaluation plan, and research roadmap for OmniAgentAI.

Keywords: multi-agent systems, agentic RAG, Tree of Thoughts, ReAct, Chain-of-Thought, neuro-symbolic AI, MCP, CrewAI, self-verification, vLLM, memory agents.

1. Introduction

Large language models can answer many natural language questions, but production AI systems often require more than fluent text generation. Real-world tasks may require external data, domain-specific tools, private databases, structured workflow execution, safety checks, memory, and validation. A single LLM call is often insufficient for tasks such as medical triage, code generation, travel booking, financial planning, or factual general knowledge queries that require source grounding.

OmniAgentAI addresses this limitation by combining neural language understanding with symbolic routing, rule-based verification, tool execution, and agent workflows. The system is designed as a tree of agents. A root router receives a query and delegates it to specialized agents when direct LLM answering is not enough. Each agent has its own reasoning policy, action tools, retrieval sources, and validation procedure.

The central research question is:

Can a neuro-symbolic multi-agent architecture improve reliability, domain coverage, and factual grounding by combining LLM reasoning, Tree-of-Thought planning, ReAct tool use, Retrieval-Augmented Generation, MCP-connected tools, CrewAI-style evaluation, and self-verification?

2. Background and Related Work

Retrieval-Augmented Generation introduced the idea of combining parametric model memory with non-parametric retrieved knowledge, improving specificity and factuality for knowledge-intensive tasks. This supports OmniAgentAI's Web RAG and fact RAG design.

Chain-of-Thought prompting showed that intermediate reasoning steps can improve performance on complex arithmetic, commonsense, and symbolic reasoning tasks. OmniAgentAI adapts this idea as staged reasoning for agents such as calculator, weather, general knowledge, and simpler task agents.

Tree of Thoughts generalizes linear reasoning into exploration over multiple candidate reasoning branches. OmniAgentAI uses this for agents where multiple strategies must be compared, such as coding tasks where Dijkstra, Bellman-Ford, Floyd-Warshall, BFS, dynamic programming, or segment trees may compete as candidate solution strategies.

ReAct combines reasoning and acting, allowing a model to interleave reasoning with external actions such as search, tool calls, API queries, or database access. OmniAgentAI follows this pattern in agents that must call tools, MCP servers, RAG systems, or domain databases.

The Model Context Protocol provides a standard way to connect AI assistants with external data systems and tools. OmniAgentAI uses MCP conceptually as the action bridge for web search, Google or browser retrieval, database tools, healthcare doctor lookup, and other external services.

vLLM and PagedAttention support efficient LLM serving. OmniAgentAI's future architecture includes vLLM to reduce latency and improve throughput when using multiple models or concurrent agent calls.

Neuro-symbolic AI combines neural perception or language understanding with symbolic rules, logical checks, and verifiable execution. OmniAgentAI follows this design: neural components classify intent and generate candidate answers, while symbolic components route tasks, enforce domain rules, validate outputs, call deterministic tools, and verify claims.

3. System Overview

OmniAgentAI is organized around the following high-level pipeline:

- 1. User query input.
- 2. Memory retrieval for prior conversation context.
- 3. LLM capability check for simple safe queries.
- 4. Router selection when direct LLM response is insufficient.
- 5. Specialized agent execution.
- 6. Reasoning through Chain-of-Thought or Tree-of-Thought.
- 7. Action through tools, MCP, Web RAG, fact RAG, normal RAG, APIs, or databases.
- 8. Observation collection.
- 9. ReAct cycles for multi-step tasks.
- 10. CrewAI-style analysis and validation.
- 11. Multi-LLM collaboration when useful.
- 12. Verification and self-correction.
- 13. Final response or retry/backtracking through router or agent node.

The system can be represented as:

User Query

```
-> Memory
-> LLM Capability Check
-> Router Root
-> Domain Agent
-> Reasoning Policy
-> Action Layer
-> Observation
-> Crew Evaluation
-> Multi-LLM Collaboration
-> Verification
-> Self-Correction
-> Final Answer or Retry
```

4. Router and Agent Tree

The router is the root node of the system. It classifies the query into domain routes. Agents are leaves or parent nodes in the tree. A parent node can contain its own internal multi-agent workflow.

Example routes include:

- CodingAgent for algorithm design, code generation, debugging, and validation.
- GeneralAgent for general knowledge, factual explanation, and Web RAG.
- CalculatorAgent for deterministic mathematical computation.
- WeatherAgent for current weather tools.
- Banking or FinanceAgent for financial tasks.
- TripPlanAgent and travel agents for flights, hotels, trains, buses, restaurants, and events.
- HealthcareAgent for symptom questioning, medical knowledge retrieval, doctor recommendation, and booking.
- ResearchAgent for literature review and research synthesis.

The router must avoid false routing. For example, "Why did the Roman Empire fall?" should be routed to GeneralAgent, not CodingAgent. This requires priority rules and factual-question detection before permissive coding classification.

5. LLM Capability Check

Before routing, OmniAgentAI can perform a direct LLM capability check:

If the query is simple, stable, safe, and answerable from model knowledge, the LLM may respond directly after verification. If the query needs fresh information, external evidence, private data, computation, booking, healthcare triage, code validation, or tool execution, the system passes the query to the router.

This design reduces unnecessary agent overhead while keeping high-risk or knowledge-sensitive tasks grounded in tools and retrieval.

6. Reasoning Layer

6.1 Chain-of-Thought Style Staged Reasoning

Some agents use a sequential reasoning structure. For safety, the system should store concise reasoning summaries or task plans rather than exposing private chain-of-thought. Example staged agents include:

- CalculatorAgent: parse expression, compute, validate numeric output.
- WeatherAgent: detect location and date, call weather API, summarize result.
- GeneralAgent: classify question, retrieve evidence, synthesize answer, verify.

6.2 Tree-of-Thought Planning

Agents with multiple competing strategies use ToT. The CodingAgent is a primary example:

Query: "Find the shortest distance."

Candidate strategies:

- Dijkstra: high score when graph has non-negative weighted edges.
- Bellman-Ford: high score when negative weights may exist.

- Floyd-Warshall: high score for all-pairs shortest paths on smaller graphs.
- BFS: high score for unweighted graphs.

The ToT planner scores candidate strategies and selects the best one based on constraints, expected complexity, input format, and validation evidence. The selected strategy is then passed to the action layer for code generation, compilation, tests, and review.

7. Action Layer

After reasoning, agents perform actions. Actions are domain-specific and may include:

- Web RAG: search web pages, chunk content, embed chunks, retrieve top context, and synthesize grounded answers.
- Fact RAG: retrieve from curated factual knowledge bases.
- Normal RAG: retrieve from local documents or uploaded files.
- MCP tools: call external APIs, browser tools, database tools, or service connectors.
- Domain databases: healthcare doctor database, booking database, finance data, product catalog, movie data, or travel data.
- Deterministic tools: calculator, compiler, validator, code runner, parser, or verifier.

For the GeneralAgent, the preferred action path is:

Query

```
-> Reasoning plan
-> Web RAG search
-> Chunking
-> Embedding
-> Vector search
-> Top context
-> LLM synthesis
-> Verifier
-> Self-correction
-> Final answer
```

This prevents the system from returning unsupported answers such as "I guess" or unverified LLM-only responses for factual questions.

8. ReAct Cycles

ReAct cycles allow the system to reason, act, observe, and decide the next step. A general loop is:

Reason: identify missing evidence. Action: call web search, RAG, MCP tool, or database. Observation: collect retrieved result. Reason: decide whether evidence is enough. Action: refine query, call another tool, or proceed.

Observation: collect additional facts. Final: synthesize and verify.

Multiple ReAct cycles are useful when a query has more than one requirement. For example, "I have fever and cough" should not immediately answer with a diagnosis. It should ask follow-up questions, retrieve medical knowledge, recommend the right specialist, and optionally book an appointment.

9. Healthcare Crew Workflow

HealthcareAgent can be structured as a parent agent with internal CrewAI nodes:

HealthcareAgent

```
-> QuestionAgent
-> Age? Temperature? Duration? Severity?
-> AnalysisAgent
-> Possible diseases or risk categories
-> RAGAgent
-> Medical knowledge retrieval
-> DoctorRecommendationAgent
-> Find specialist
-> BookingAgent
-> Appointment booking
```

This workflow reduces hallucination by separating questioning, analysis, retrieval, recommendation, and booking. It also allows the system to enforce healthcare safety rules, such as recommending emergency care for severe symptoms.

10. CrewAI Evaluation and Multi-Agent Review

After one or more ReAct loops, outputs pass through a CrewAI-style evaluation layer. This layer can include:

- AnalyzerAgent: checks whether the answer addresses the query.
- ValidatorAgent: verifies data consistency and task completion.
- SafetyAgent: checks harmful or unsafe recommendations.
- CriticAgent: identifies missing evidence or weak claims.
- DomainExpertAgent: applies domain-specific rules.

For coding, the review may include compiler checks, unit tests, complexity analysis, template detection, and problem-statement validation. For healthcare, the review may include symptom safety, evidence grounding, and doctor-specialty matching.

11. Multi-LLM Collaboration and vLLM

OmniAgentAI can use multiple LLMs in parallel, such as GPT, Claude, Qwen, DeepSeek, Phi, or local models served through vLLM. The system can ask each model for a role-specific judgment:

- GPT: general synthesis and instruction following.
- Claude: critique, safety, and long-context analysis.
- Qwen: coding or multilingual reasoning.
- DeepSeek: code and mathematical reasoning.
- Phi or smaller models: fast local checks.

The system aggregates these outputs through voting, confidence scoring, or verifier-based selection. vLLM can reduce inference latency by batching and serving multiple requests efficiently.

12. Verification and Self-Correction

The final response is not returned until it passes verification. Verification may include:

- Source support from retrieved documents.
- Cross-source consistency.
- Rule-based sanity checks.
- Domain validation.
- Test execution for code.
- Database confirmation for booking or doctor details.
- Calculator recomputation for numeric outputs.

If verification succeeds, the answer is returned. If verification fails, the system either self-corrects, retrieves more evidence, calls a different tool, or routes back to the relevant agent node. This creates a retry cycle:

Draft answer

```
-> Verify
-> If valid: respond
-> If invalid: self-correct or return to action/router
-> Repeat until validated or safe fallback
```

13. Memory and Uploaded File Support

The MemoryAgent stores prior user queries, assistant answers, and conversation context. It supports follow-up queries such as:

User: "What is artificial intelligence?" Assistant: gives answer. User: "Give more details."

The system resolves "more details" to the prior topic and generates an expanded response. Memory also supports personalization, task continuity, and long-running workflows.

The upload layer supports PDFs, DOCX files, spreadsheets, and other documents. Uploaded documents can be chunked, embedded, indexed, and retrieved through normal RAG. This allows the system to answer questions using user-provided files rather than only web data or built-in facts.

14. Neuro-Symbolic Design

OmniAgentAI is neuro-symbolic because it combines:

Neural components:

- Natural language understanding.
- Query classification.
- Semantic retrieval.
- Candidate answer generation.
- Multi-model synthesis.
- Pattern recognition over user intent.

Symbolic components:

- Router rules.
- Tool schemas.
- Domain workflows.
- Validation policies.
- Database constraints.
- Algorithm complexity checks.
- Safety rules.
- Verification thresholds.

This hybrid design aims to use the flexibility of neural models while preserving the reliability of symbolic execution and validation.

15. Example: General Knowledge Query

Query: "Why did the Roman Empire fall?"

Expected OmniAgentAI flow:

- Search political instability. - Search economic decline. - Search military overextension. - Search external invasions. - Summarize with source grounding.

- 1. LLM recognizes this as a historical explanation query.
- 2. Router sends it to GeneralAgent.
- 3. ToT planner creates a plan:
- 4. Web RAG retrieves relevant sources.
- 5. LLM generates a source-grounded answer.
- 6. Verifier checks support across retrieved documents.
- 7. Self-correction removes unsupported claims.

- 8. Final answer explains political instability, economic pressure, military problems, administrative division, and invasions.

16. Example: Coding Query

Query: "Find the shortest distance in a weighted graph."

Expected CodingAgent flow:

- 1. Query classifier detects coding/algorithm task.
- 2. ToT planner generates candidate algorithms.
- 3. Candidate scoring selects Dijkstra if edge weights are non-negative.
- 4. RAG retrieves algorithm notes and C++ templates.
- 5. Code generator creates implementation.
- 6. Compiler checks syntax.
- 7. Unit tests validate behavior.
- 8. Complexity analyzer reports time and memory.
- 9. CrewAI validator checks final answer.
- 10. Self-correction retries if tests fail.

17. Research Contributions

The proposed architecture contributes:

- 1. A tree-structured multi-agent router that delegates tasks to specialized parent and leaf agents.
- 2. A hybrid reasoning policy using Chain-of-Thought style staged reasoning for simpler agents and Tree-of-Thought planning for strategy-search agents.
- 3. An action layer that unifies MCP, Web RAG, normal RAG, domain tools, APIs, databases, and deterministic validators.
- 4. A ReAct-based loop for multi-step tasks requiring observation and tool interaction.
- 5. CrewAI-style multi-agent evaluation for hallucination reduction.
- 6. Multi-LLM collaboration for speed, diversity, and robustness.
- 7. A neuro-symbolic verification layer combining neural synthesis with symbolic rules and deterministic checks.
- 8. Memory and document-upload support for contextual continuity.
- 9. A roadmap toward autonomous research systems, theorem proving, multimodal reasoning, and algorithm discovery.

18. Evaluation Plan

OmniAgentAI can be evaluated across several dimensions:

- Routing accuracy: percent of queries sent to the correct agent.
- Answer factuality: source-supported correctness for general knowledge.
- Tool success rate: successful MCP, API, database, and RAG calls.
- Hallucination rate: unsupported claims after verification.
- Coding success: compile rate, test pass rate, algorithm-selection accuracy.
- Healthcare safety: correct follow-up questioning and specialist routing.
- Latency: response time with and without vLLM batching.
- Memory quality: follow-up resolution accuracy.
- User satisfaction: task completion and clarity.

Suggested datasets:

- General QA benchmarks for factual RAG.
- Coding benchmarks for algorithm selection and code correctness.
- Synthetic routing datasets across all agents.
- Healthcare triage simulations with safety labels.
- Uploaded-document QA tests.

19. Future Work

Future OmniAgentAI work includes:

- Multimodal agents for image, audio, video, and document understanding.
- Formal theorem proving with Lean for verifiable math and logic.
- Algorithm discovery agents that propose and test new algorithms.
- AI scientist workflows that generate hypotheses, run experiments, write papers, and review results.
- Distributed agent ecosystems where multiple machines or services collaborate.
- Autonomous research systems that perform literature review, experimentation, evaluation, and publication drafting.
- Stronger MCP security auditing and sandboxing.
- Better verifier agents for cross-domain factuality and safety.

20. Conclusion

OmniAgentAI proposes a practical and extensible architecture for agentic AI systems. Instead of relying on one LLM response, it combines direct model answering, router-based delegation, specialized agents, Tree-of-Thought planning, Chain-of-Thought style staged reasoning, ReAct tool use, RAG, MCP, CrewAI evaluation, multi-LLM collaboration, memory, uploaded-file retrieval, verification, and self-correction. The result is a neuro-symbolic

design intended to improve correctness, reduce hallucination, support complex workflows, and prepare for future autonomous research systems.

References

- 1. Lewis, P., Perez, E., Piktus, A., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." arXiv:2005.11401, 2020. https://arxiv.org/abs/2005.11401
- 2. Wei, J., Wang, X., Schuurmans, D., et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." arXiv:2201.11903, 2022. https://arxiv.org/abs/2201.11903
- 3. Yao, S., Zhao, J., Yu, D., et al. "ReAct: Synergizing Reasoning and Acting in Language Models." arXiv:2210.03629, 2022. https://arxiv.org/abs/2210.03629
- 4. Yao, S., Yu, D., Zhao, J., et al. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." arXiv:2305.10601, 2023. https://arxiv.org/abs/2305.10601
- 5. Wang, L., Ma, C., Feng, X., et al. "A Survey on Large Language Model based Autonomous Agents." arXiv:2308.11432, 2023. https://arxiv.org/abs/2308.11432
- 6. Anthropic. "Introducing the Model Context Protocol." 2024. https://www.anthropic.com/news/model-context-protocol
- 7. Kwon, W., Li, Z., Zhuang, S., et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." 2023. https://arxiv.org/abs/2309.06180
- 8. Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery." arXiv:2408.06292, 2024. https://arxiv.org/abs/2408.06292
- 9. Tang, X. "A Comprehensive Survey of the Lean 4 Theorem Prover: Architecture, Applications, and Advances." arXiv:2501.18639, 2025. https://arxiv.org/abs/2501.18639
- 10. Hasan, M. M., Li, H., Fallahzadeh, E., Adams, B., and Hassan, A. E. "Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers." arXiv:2506.13538, 2025. https://arxiv.org/abs/2506.13538